

Botmakers Assignment

➤ Introduction to React

1) What is React?

React is a JavaScript library used for create front end development it helps developers build dynamics, reusable and interactive user interface.

2) Why was React created?

Back in 2011 Facebook developer faced a serious issue know as the “Ghost Problem” or we can say reloading, On the web page whenever user did small action ex. User clicks like button entire web page get reload. to encounter this problem, they create React.

3) Problems React solves

React solved reloading problem introducing virtual DOM its light weight copy of the actual DOM.

4) Features of React?

Most important feature of react is a virtual DOM, component-based architecture, JSX, one way data binding (parent to child), react Hooks.

5) Advantages and disadvantages

Advantages

SPA, VDOM, JSX, One-way data binding

Disadvantages

- Uses more memory because of the Virtual DOM.
- Needs additional libraries for routing, state management, and advanced features.
- Handles only the UI layer, not the entire application logic.

6) Why companies use React

- Development Speed and Reusability
- Easier Maintenance for Large Codebases
- High Performance at Scale
- Cross-Platform Development (React Native)
- Huge Community

7) Popular companies using React

- Facebook, Instagram, Netflix, and Airbnb

Botmakers Assignment

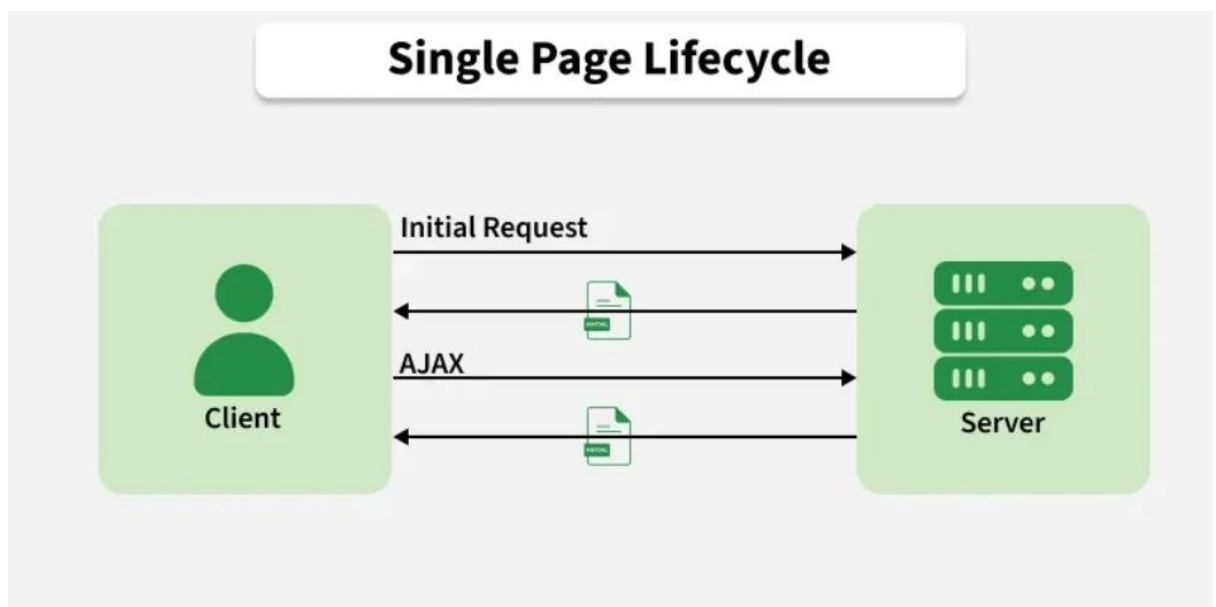
8) React Ecosystem

<https://dev.to/avinashvagh/react-ecosystem-in-2024-418k>

➤ React Architecture

1) SPA (Single Page Application)

A single page application is web app that reload a whole page without refreshing. It uses JavaScript to update only the required part of the screen based on the user action. Sends and update the only required data from the server and also provide more responsive and seamless interface



2) Component-Based Architecture

A component-based architecture gives us reusability and flexibility in block code. We don't need to create them again and again; we can just create them once and use them wherever we want in our web application. React is a fully component-based architecture library. It gives us scalability. It helps developers ship work fast, and it is also easy to maintain the application code-base.

3) Virtual DOM

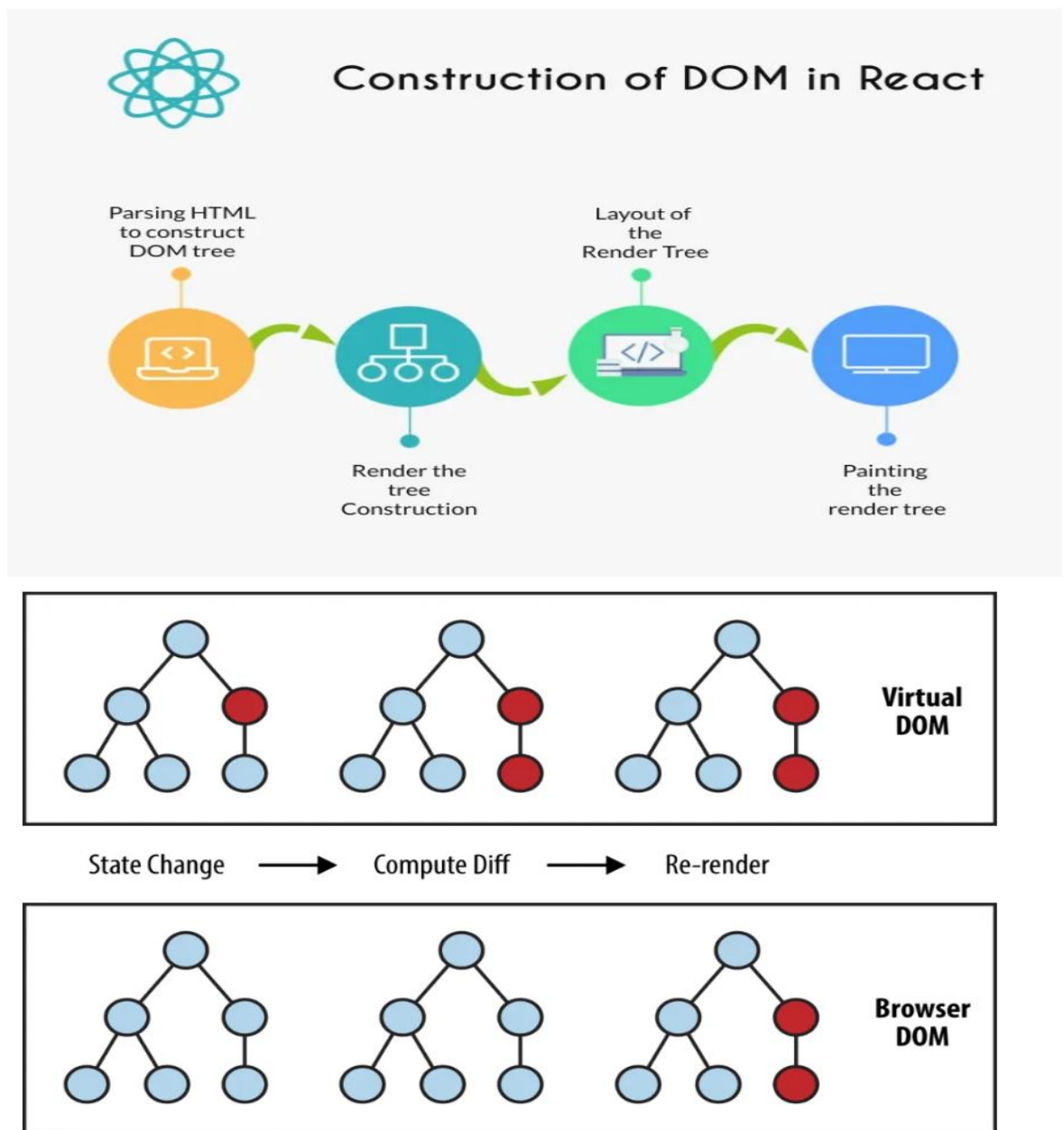
A **Virtual DOM** is basically a lightweight copy of the Real DOM. It has fewer properties than the actual DOM, so working with it is faster and more efficient. Whenever something changes in our application, React creates a new Virtual DOM and compares it with the previous one. It finds out what exactly has changed and then updates only those parts in the Real DOM instead of updating the whole page.

Botmakers Assignment

4) Real DOM

The Real DOM is heavy because it has many properties and elements. If we make changes directly to it, the browser has to do more work to update the UI, which can affect performance.

5) Rendering Process



Blog Link: <https://nikshindeblogs.medium.com/react-reconciliation-547024dda9fc>

Botmakers Assignment

6) Diffing Algorithm

The **Diffing Algorithm** is basically used to find the difference between the old and new Virtual DOM. It checks what has changed and updates only those parts in the Real DOM. We can understand it with a simple example of **Git**. Git keeps track of the changes we make in our code. In the same way, the Diffing Algorithm keeps track of the changes between the old and new Virtual DOM and updates only the necessary parts in the Real DOM.

7) Reconciliation

Reconciliation is basically the process of finding out what has changed between the old and new Virtual DOM. React then makes only the necessary changes to the Real DOM instead of updating everything.

➤ JSX

1) What is JSX?

JSX stands for JavaScript XML or JavaScript Syntax Extension. It allows us to write HTML-like code directly inside JavaScript, and it is commonly used with React.js. JSX is a syntactical sugar that allows us to write UI code in a way that looks similar to HTML but works like JavaScript. JSX is transformed into React elements, which React uses to create and update the UI. We can also create custom components/tags in JSX because JSX supports an XML-like syntax, allowing us to define and use our own components.

2) Why JSX is used?

JSX gives us the flexibility to write UI logic and the JavaScript logic that drives the UI in the same file. This makes the code more readable, maintainable, and easier to understand, because the structure and behaviour of a component can be kept together.

3) Expressions inside JSX

In JSX, we can use JavaScript expressions inside curly braces `{}`. For example, we can use already-declared variables, perform calculations, call functions, or use conditions that return a value. We cannot write JavaScript statements like `if`, `for`, or `const` directly inside JSX. If we need to declare a variable or use a statement, we should do it outside the JSX and then use the result inside `{}`.

Botmakers Assignment

4) Fragments

in React are used when we want to return multiple elements from a component without adding an extra HTML element to the DOM. fragment itself does not create an extra element in the DOM.

5) ClassName

className is used in React to apply CSS classes to HTML elements. It works similarly to the class attribute in normal HTML, but in JSX we use className because class is a reserved keyword in JavaScript.

6) HtmlFor

htmlFor is used in React to connect a <label> with an HTML form element like an <input>. In normal HTML, we use the for attribute, but in JSX we use htmlFor because for is a reserved keyword in JavaScript.

7) Self-closing tags

In JSX, elements that don't have children should be written as self-closing tags using />

8) Conditional Rendering in JSX

Conditional rendering is the process of displaying different UI elements based on a condition.

9) JSX Rules

JSX follows a few rules that are different from normal HTML, such as using className, htmlFor, curly braces for JavaScript expressions, and properly closing all tags.

- **Return a single parent element**

A component should return one parent element. We can use a <div> or a Fragment <>...</>

- **Use className instead of class**

In JSX, we use className to apply CSS classes.

- **Use htmlFor instead of for**

When connecting a <label> with an input, we use htmlFor

Botmakers Assignment

- **Close all tags**

Every JSX element must be properly closed. For example, `<input />`, `<img />`, and `<br />`.

- **Use curly braces for JavaScript expressions**

We can use JavaScript expressions inside `{}`

```
const name = "Ravi";  
return <h1>Hello {name}</h1>
```

- **Use camelCase for most attributes**

For example, `onClick`, `onChange`, and `tabIndex` instead of HTML-style `onclick`, `onchange`, and `tabindex`.

- **JavaScript statements cannot be written directly inside JSX**

We can use expressions inside JSX, but statements like `if`, `for`, and `const` cannot be directly written inside `{}`.

Botmakers Assignment

1) What is React?

Definition: React is a popular, free JavaScript tool used for building user interfaces (the visible parts of a website).

Explanation: Imagine building a house entirely out of Lego blocks. Instead of molding one giant plastic house, you snap together small, reusable blocks (like windows, doors, and walls). React lets you build websites exactly like this: by snapping together smaller code blocks called "components".

Example: When you log into Facebook or Instagram, the entire website is built using React. The search bar is one Lego block, the post feed is another, and the chat window is another.

2) Why React?

Definition: React is chosen by developers because it is incredibly fast, easy to maintain, and backed by a massive community.

Explanation: Before React, if you clicked "Like" on a post, the web browser sometimes had to reload the whole page just to show the heart turning red. React is smart; it only updates the tiny piece of the screen that changed, leaving everything else alone. This makes websites feel as fast and smooth as phone apps.

Example: Scrolling through a never-ending social media feed without the page ever refreshing or stuttering.

3) Virtual DOM

Definition: A lightweight, invisible copy of the actual webpage kept in the computer's memory.

Explanation: Think of the Virtual DOM as a draft or a blueprint of a room. If you want to move a sofa, it is much faster to erase the sofa on the blueprint and redraw it, rather than moving a heavy real sofa just to see how it looks. React updates the "blueprint" first, compares it to the old blueprint, and then makes only the exact changes needed to the "real house."

Example: If only a clock text changes from 10:00 to 10:01, React finds exactly that text in its blueprint and updates just those few pixels on screen.

Botmakers Assignment

4) Real DOM

Definition: The actual, physical structure of the webpage that the browser draws on your screen (Document Object Model).

Explanation: The Real DOM is the actual house built from the blueprint. Updating it is "heavy" and slow. If you tell the Real DOM to change something without React, it often forces the browser to redraw large parts of the house all over again.

Example: The standard HTML you see when you right-click a webpage and click "Inspect".

5) JSX (JavaScript XML)

Definition: A special syntax that lets us write HTML-like code directly inside our JavaScript code.

Explanation: Usually, HTML (for looks) and JavaScript (for logic) live in separate files. JSX brings them together. It allows you to design your website visually right next to the logic that controls it. It looks like HTML, but under the hood, React translates it into regular JavaScript.

Example:

```
9
10  const greeting = <h1>Hello, world!</h1>;
11
```

6) Components

Definition: Independent, reusable pieces of code that represent a part of the user interface.

Explanation: Think of a component as a custom HTML tag. If you need a shiny blue button in 10 different places on your website, you don't write the code 10 times. You create one `` component and use it 10 times.

Example:

```
1
2  const App = ()=>{
3    return(
4      <>
5        <h1> Hello! This is a JSX </h1>
6      </>
7    )
8  }
9
```


Botmakers Assignment

7) Props (Properties)

Definition: A way to pass data from a parent component down to a child component.

Explanation: Props are like giving instructions to your Lego blocks. If you have a `` component, you can pass it a "name" prop and an "age" prop so it knows whose information to display. Props are read-only; the child cannot change the instructions it was given.

Example:

```
11 // Passing a prop
12 <App
13   firstName="Ravi"
14   lastName="Gahire"
15 />
```

8) State

Definition: A component's personal memory. It holds data that can change over time.

Explanation: While Props are instructions passed from the outside, State is a component's internal secret notebook. If you have a "Like" button, it uses its "State" to remember if it has been clicked or not. Whenever a component's state changes, React automatically redraws that component on the screen.

Example:

```
1 import { useState } from 'react';
2
3 function Counter() {
4   const [count, setCount] = useState(0); // State starts at 0
5   return <div>You clicked {count} times</div>;
6 }
7
```

Botmakers Assignment

9) Event Handling

Definition: Catching and responding to user actions, like mouse clicks, typing, or hovering.

Explanation: Websites need to be interactive. In React, you connect user actions to functions. It looks very similar to standard HTML, but React uses camelCase naming (like `onClick` instead of `onclick`).

Example:

```
1 function Button() {  
2   function handleClick() {  
3     alert('Button was clicked!');  
4   }  
5   return <button onClick={handleClick}>Click Me!</button>;  
6 }
```

10) Conditional Rendering

Definition: Showing different components or elements on the screen based on a specific condition.

Explanation: It's just like making a decision in real life. "IF it is raining, take an umbrella. OTHERWISE, take sunglasses." In React: "IF the user is logged in, show the Dashboard. OTHERWISE, show the Login screen."

Example:

```
1 function App({ isLoggedIn }) {  
2   return (  
3     <div>  
4       {isLoggedIn ? <h1>Welcome back!</h1> : <h1>Please sign in.</h1>}  
5     </div>  
6   );  
7 }  
8
```

Botmakers Assignment

11) Lists & Keys

Definition: Using arrays to render multiple similar items, and giving each item a unique identifier (key).

Explanation: When you want to display a list of groceries, you don't write 100 separate lines of code. You loop through your list using a tool called `.map()`. React asks you to give every item in that list a unique "key" (like a barcode) so if you delete or move an item, React knows exactly which one it is without getting confused.

Example:

```
1  const groceries = ['Apples', 'Bananas', 'Milk'];
2
3  function GroceryList() {
4    return (
5      <ul>
6        {groceries.map((item, index) => (
7          <li key={index}>{item}</li>
8        ))}
9      </ul>
10    );
11  }
```

12) Forms

Definition: Managing user input fields (like text boxes and checkboxes). **Explanation:** In standard HTML, forms remember what you typed all by themselves. In React, we prefer React to be in charge. We link the input box to React's State. Whatever you type updates the State, and the State updates the box. This is called a "controlled component."

Example:

```
1  function NameForm() {
2    const [name, setName] = useState('');
3
4    return (
5      <input
6        type="text"
7        value={name}
8        onChange={(e) => setName(e.target.value)}
9        placeholder="Type your name..."
10      />
11    );
12  }
13
```

Botmakers Assignment

13) Folder Structure

Definition: How we organize our files and folders inside a React project. **Explanation:** A messy desk makes it hard to work. React doesn't force you into one specific folder setup, but most developers organize things by separating visual pieces (components) from background images (assets) and full screens (pages).

Example:

```
my-react-app/  
├─ public/      # Public files like index.html  
├─ src/         # Your working folder  
│  ├─ assets/   # Images and icons  
│  ├─ components/ # Reusable Lego blocks (Button.js, Header.js)  
│  ├─ pages/    # Full pages (Home.js, About.js)  
│  └─ App.js    # The main root component
```

14) Advantages of React

Definition: The core benefits that make React incredibly popular.

Explanation: 1. Speed: Because of the Virtual DOM, it loads and updates incredibly fast. 2. Reusability: Write a component once, use it everywhere (saves time). 3. Huge Community: If you get stuck, someone on the internet has already solved your problem.

Example: Companies like Netflix, Airbnb, and Uber rely on React to handle millions of daily users.

15) Limitations of React

Definition: The downsides or missing pieces when using React.

Explanation: 1. It's just UI: React only handles what the user sees. You still need to find other tools to handle URLs (routing) or talk to databases. 2. Steep Learning Curve: For pure beginners, combining HTML with JavaScript (JSX) can feel weird at first. 3. Fast-Paced: The tools update so frequently that online tutorials can become outdated quickly.

Example: Building a complete store requires React (for visuals) + React Router (for links) + Redux (for complex state) + a backend server

16) React Best Practices

Definition: The "golden rules" professional developers follow to keep their code clean and bug-free.

Explanation:

1. Keep Components Small: If a component is doing too much, break it into smaller components.

Botmakers Assignment

2. Use Functional Components: Stick to modern React hooks instead of old-fashioned "Class" components.

3. Don't Modify State Directly: Never write `state = "new value"`. Always use the setter function provided!

4. Name files clearly: Use PascalCase (like `UserProfile.js`) for your component files.

Example: Instead of having a single 1000-line `App.js` file, separate your code into `Header.js`, `Footer.js`, and `MainContent.js`.